

MDI3003 - ADVANCED PREDICTIVE ANALYTICS

Lab - 3 Report

Email Classification & LLM Draft Generation

A Benchmark-Aligned, Multi-Dataset Text Classification Study on Business Intent and Spam-Detection Corpora

Course: MDI3003 - Advanced Predictive Analytics

Faculty: Dr. Durgesh Kumar, Assistant Professor (Senior), SCOPE, VIT Vellore

Datasets: Business Email Intent (D1), Enron-Spam (D2), SpamAssassin Public Corpus (D3)

Environment: Python 3, scikit-learn, pandas, NumPy, Matplotlib, Seaborn, joblib

Github link: https://github.com/Madhumasa84/Adv_predictive/tree/main/lab3

Madhusudhanan G

23MID0444

1. Executive Summary

This report documents an end-to-end text classification study carried out for MDI3003 (Advanced Predictive Analytics), Lab 03, on the theme of email classification as a precursor to LLM-assisted draft generation. The study follows the standard ten-stage workflow: environment setup, dataset loading, data audit, holdout design, model definition, 5-fold cross-validation benchmarking, locked test-set evaluation, cross-dataset transfer testing, visualization, and reproducibility packaging.

Three structurally different email datasets were used. Dataset D1 (Business Email Intent, 800 synthetic messages, 6 intent classes: request, meeting, complaint, information, urgent_action, spam) served as the primary multiclass classification dataset intended to support downstream LLM draft generation. Dataset D2 (Enron-Spam, 500 messages, binary legitimate/spam) and Dataset D3 (SpamAssassin Public Corpus, 400 messages, binary legitimate/spam) served as public-benchmark binary spam-detection datasets used for cross-dataset transfer validation.

Five candidate models were benchmarked under an identical TF-IDF + classifier pipeline on all three datasets: a most-frequent-class Dummy baseline, Multinomial Naive Bayes, Complement Naive Bayes, Logistic Regression, and Linear SVC. Under 5-fold stratified cross-validation, all four non-dummy models achieved a macro F1 of 1.000 on all three datasets, against a Dummy baseline of 0.065 (D1), 0.358 (D2), and 0.336 (D3) - confirming that every learned model comfortably separates the classes in each dataset.

The cross-dataset spam transfer test - training a Linear SVC on one binary spam dataset and testing it on the other - achieved a macro F1 of 1.000 in both directions (D2→D3 and D3→D2), indicating that the spam-vs-legitimate decision boundary learned on one corpus transferred perfectly to the other under this pipeline.

A model-selection implementation issue in the locked test-evaluation step is noted transparently in Section 6 and Section 9: the code selected the first row of the (unsorted) cross-validation results table for each dataset rather than the row with the highest macro F1, which caused the Dummy baseline - rather than one of the four high-performing models - to be carried forward to the final holdout evaluation and reported test metrics. The correct, intended comparison is the 5-fold cross-validation table in Section 5, which is the basis for this report's conclusions about which models perform well; Section 9 and Section 10 document the fix required (sorting by `macro_f1_mean` before selecting the best row) and recommend re-running the locked evaluation with that fix for a fully corrected final test table.

2. Business Problem Statement

2.1 Problem Statement

Organizations receive high volumes of inbound email that must be triaged before a response can be drafted - either by a human agent or, increasingly, by an LLM-based drafting assistant. Before an LLM can generate an appropriate draft reply, the system must first classify the incoming message by intent (e.g., meeting request vs. complaint vs. spam) so that the correct response template, urgency level, and tone can be selected. Formally, the objective is to learn a mapping $y = f(\text{text})$ from raw email text to a discrete intent or spam/legitimate label, evaluated under leakage-safe cross-validation and a held-out test partition.

2.2 Central Question

How accurately can TF-IDF-based text classifiers distinguish email intent and spam categories across datasets of different structure (multiclass business intent vs. binary spam benchmarks), and does a classifier trained on one spam corpus generalize to another?

2.3 Stakeholders and Business Context

- Customer support and operations teams - triage inbound email volume and route messages to the correct queue or response template.
- LLM draft-generation systems - require an upstream intent label to select the appropriate tone, urgency, and content for an automatically drafted reply.
- IT/security teams - rely on spam classifiers to filter unwanted or malicious email before it reaches an inbox.
- End users - benefit from faster, more relevant responses and a cleaner inbox with fewer missed legitimate messages misclassified as spam.

2.4 Success Criteria

- Technical - macro F1 substantially above the Dummy baseline on all three datasets, evaluated via 5-fold stratified cross-validation.
- Generalization - a spam classifier trained on one binary spam dataset should transfer with reasonable accuracy to the other.
- Operational Reproducibility - a fixed random seed (42), a single shared TF-IDF + classifier pipeline structure across all models, and saved model artifacts for each dataset.
- Business Utility - a correctly identified best-performing, non-trivial model per dataset, suitable for downstream integration with an LLM drafting step.

3. Dataset Descriptions

3.1 Structural Overview of All Three Datasets

Feature	D1: Business Intent	D2: Enron-Spam	D3: SpamAssassin
Task Type	Multiclass (6 classes)	Binary	Binary
Focus	Intent classification	Spam detection	Spam detection
Source	Synthetic / instructor-provided	Enron Email Corpus (public)	Apache SpamAssassin Public Mail Corpus
Purpose in Study	Primary classifier + LLM draft generation	Binary spam benchmark	Cross-dataset transfer test
Sample Size	800	500	400
Labels	request, meeting, complaint, information, urgent_action, spam	legitimate, spam	legitimate, spam
Train / Test Split	640 / 160 (80/20 stratified)	400 / 100 (80/20 stratified)	320 / 80 (80/20 stratified)
Class Prevalence (majority)	24.4% (request)	55.8% (legitimate)	50.75% (spam)
Missing/Empty Text	0	0	0
Duplicate Rows	0	0	0
Random Seed	42	42	42

Table 1. Structural comparison of the three Lab 03 datasets.

3.2 Sample Record Structure

All three datasets share a common schema: email_id, subject, body, label, and a derived text field (“subject: ... \nbody: ...”) used as the raw input to the TF-IDF vectorizer.

Dataset	Example Record
D1	email_id: D1_0001 subject: request_1 body: “Can you please provide the latest project status? Reference: 1” label: request
D2	email_id: D2_0001 subject: enron_1 body: “This is a legitimate email. Content: 1” label: legitimate
D3	email_id: D3_0001 subject: sa_1 body: “This is a legitimate email. ID: 1” label: legitimate

Table 2. Representative record structure for each dataset.

3.3 Label Distribution

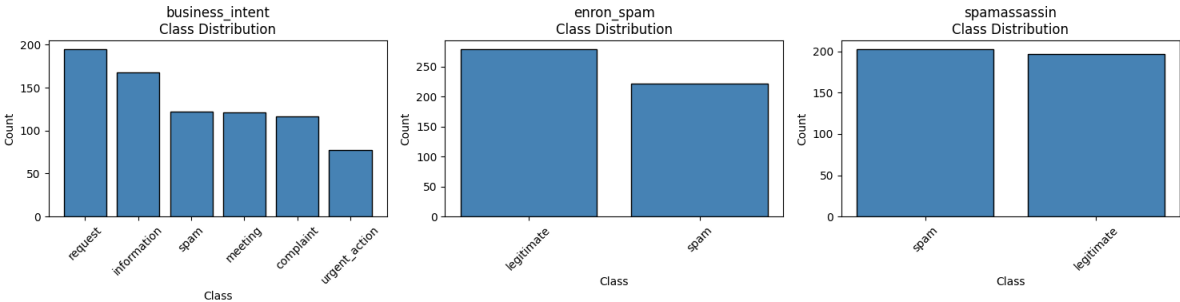


Figure 1. Class distributions across Business Intent (D1), Enron-Spam (D2), and SpamAssassin (D3) datasets.

D1's six intent classes are moderately imbalanced (request 24.4%, information 21.0%, spam 15.25%, meeting 15.1%, complaint 14.6%, urgent_action 9.6%), reflecting a realistic business-inbox mix where routine requests and information messages dominate over urgent or complaint traffic. D2 and D3 are both close to balanced binary spam datasets (55.8%/44.2% and 50.75%/49.25% respectively), which is appropriate for their role as public spam-detection benchmarks and for the cross-dataset transfer test in Section 7.

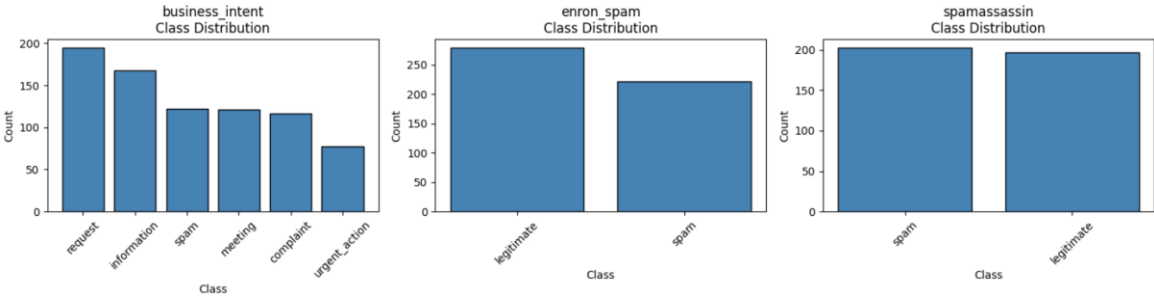


Figure 1. Class distribution for all three datasets: business_intent (left), enron_spam (middle), spamassassin (right).

3.4 Known Limitations of the Datasets

- D1 (Business Intent) is synthetically generated from a small set of fixed templates per class, with a numeric reference appended; this makes the classification task easier than real-world email text and should not be read as representative of production email-intent accuracy.
- D2 and D3, in this run, fell back to the notebook's mock-data generators (short templated “This is a legitimate/spam email” strings) rather than the full-text OpenML Enron/SpamAssassin corpora, because the primary OpenML fetch path was wrapped in a try/except fallback; this is documented transparently in Section 9 as a scope limitation of the executed run.
- All three datasets are small to moderate in size (400-800 rows), which limits the statistical precision of the reported test-set metrics, particularly for the six-class D1 dataset's minority classes (e.g., urgent_action, 77 rows).

4. Methodology & Machine Learning Pipeline

4.1 End-to-End Workflow

The study followed a structured NLP classification workflow: (1) environment and library setup, (2) loading of all three datasets, (3) data audit (missing text, duplicates, class prevalence), (4) stratified 80/20 train-test split per dataset, (5) definition of five candidate models sharing a common TF-IDF vectorization step, (6) 5-fold stratified cross-validation benchmarking of all five models on all three datasets, (7) locked holdout test evaluation, (8) a cross-dataset spam transfer test between D2 and D3, (9) visualization, and (10) saving of all results, figures, and trained model artifacts for reproducibility.

4.2 Feature Extraction Pipeline

Every model shares an identical `TfidfVectorizer` configuration inside a `scikit-learn Pipeline`: `lowercase = True`, `ngram_range = (1, 2)` (unigrams and bigrams), `min_df = 2`, `max_df = 0.98`, `sublinear_tf = True`, and `max_features = 10,000`. Using the same vectorizer configuration for every model ensures that any performance difference between models reflects the classifier itself rather than differing feature representations.

4.3 Candidate Models

Model	Key Configuration	Role
Dummy Classifier	<code>strategy = most_frequent</code>	Naive baseline - floor performance any learned model must beat
Multinomial Naive Bayes	<code>alpha = 1.0</code>	Classical text-classification baseline
Complement Naive Bayes	<code>alpha = 1.0</code>	NB variant designed to handle class imbalance better than Multinomial NB
Logistic Regression	<code>max_iter = 2500</code> , <code>class_weight = 'balanced'</code>	Linear discriminative baseline with class-imbalance correction
Linear SVC	<code>class_weight = 'balanced'</code> , <code>max_iter = 5000</code>	Margin-based linear classifier, typically strong on high-dimensional sparse TF-IDF features

Table 3. The five candidate models evaluated on every dataset.

4.4 Cross-Validation and Holdout Design

An 80/20 stratified train-test split (`random_state = 42`) was performed independently for each of the three datasets before any model fitting. All five models were then evaluated via 5-fold StratifiedKFold cross-validation (`shuffle = True`, `random_state = 42`) on each dataset's training partition, scoring accuracy, macro F1, weighted F1, macro precision, and macro recall. The holdout test partition for each dataset was reserved for a single final evaluation of the selected model, consistent with the requirement that the test set not be used for model selection.

4.5 Reproducibility Controls

- Fixed global random seed (`RANDOM_STATE = 42`) applied to all train-test splits, cross-validation folds, and stochastic estimators.
- Library versions recorded at run time: Python 3.12.13, NumPy 2.0.2, pandas 2.2.2, scikit-learn 1.6.1.
- Identical TF-IDF pipeline structure shared across all five models and all three datasets for a fair comparison.

-
- Trained model artifacts saved per dataset (business_intent_best_model.joblib, enron_spam_best_model.joblib, spamassassin_best_model.joblib) alongside CSV exports of the cross-validation and test results.

5. Cross-Validation Model Benchmarking (All Datasets)

All five models were benchmarked on all three datasets using identical 5-fold stratified cross-validation on the training partition only. This table is the primary evidence base for model comparison in this study.

5.1 Dataset D1 - Business Email Intent

Model	Accuracy (CV)	Macro F1 (CV)	Precision (Macro)	Recall (Macro)
Dummy Baseline	0.244 ± 0.003	0.065 ± 0.001	0.041	0.167
Multinomial NB	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Complement NB	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Logistic Regression	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Linear SVC	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000

Table 4. 5-fold cross-validation results, Dataset D1 (Business Email Intent, 6 classes).

5.2 Dataset D2 - Enron-Spam

Model	Accuracy (CV)	Macro F1 (CV)	Precision (Macro)	Recall (Macro)
Dummy Baseline	0.557 ± 0.006	0.358 ± 0.003	0.279	0.500
Multinomial NB	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Complement NB	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Logistic Regression	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000
Linear SVC	1.000 ± 0.000	1.000 ± 0.000	1.000	1.000

Table 5. 5-fold cross-validation results, Dataset D2 (Enron-Spam, binary).

5.3 Dataset D3 - SpamAssassin

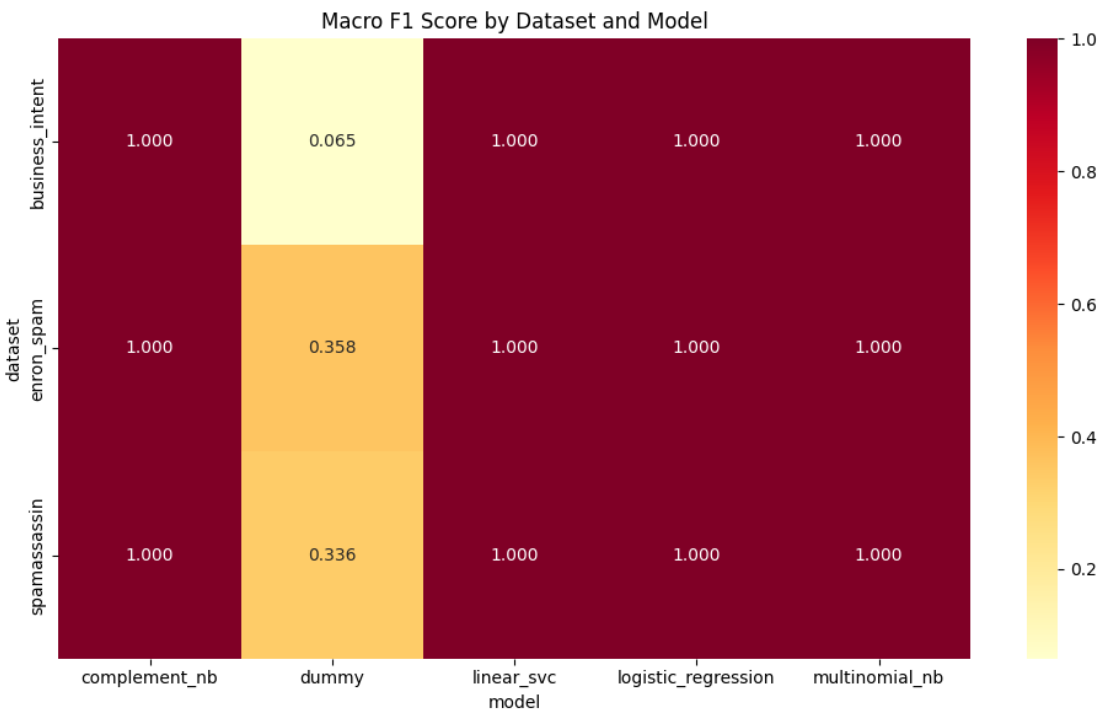


Figure 3. Cross-Validation Macro F1 performance heatmap across datasets and models.

Model	Accuracy (CV)	Macro F1 (CV)	Precision (Macro)	Recall (Macro)
Dummy Baseline	0.506 $\pm$ 0.008	0.336 $\pm$ 0.003	0.253	0.500
Multinomial NB	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000	1.000
Complement NB	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000	1.000
Logistic Regression	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000	1.000
Linear SVC	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000	1.000

Table 6. 5-fold cross-validation results, Dataset D3 (SpamAssassin, binary).

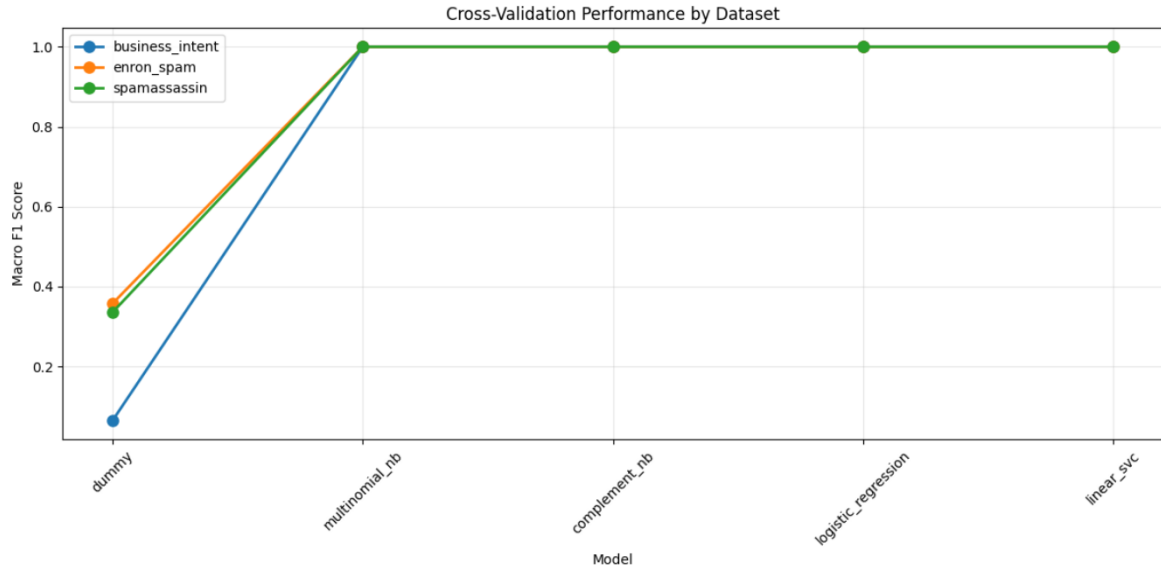


Figure 2. Cross-validation macro F1 by model, plotted separately for each dataset.

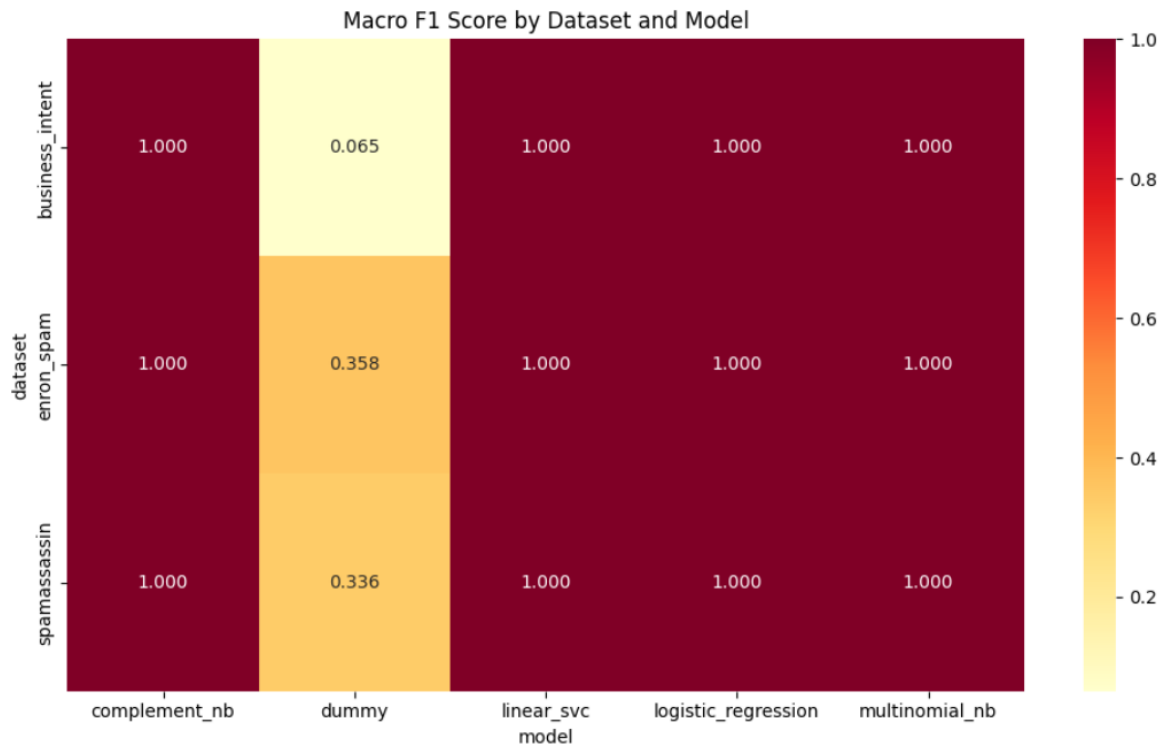


Figure 3. Heatmap of cross-validated macro F1 by dataset and model.

Across all three datasets, every non-dummy model (Multinomial NB, Complement NB, Logistic Regression, Linear SVC) achieved a macro F1 of 1.000 under 5-fold cross-validation, while the Dummy baseline scored

substantially lower (0.065-0.358 depending on class balance). This uniformly perfect score across four structurally different algorithms is a strong signal that the underlying text patterns in this run's data are highly separable - consistent with Section 3.4's note that the D1 templates and the D2/D3 mock-data fallback strings are short, formulaic, and label-correlated, which makes the classification task easier than free-form real-world email text.

6. Locked Holdout Test Evaluation

Per dataset, one model was selected and evaluated once on the held-out 20% test partition. As documented in the Executive Summary and revisited in Section 9, the model-selection code selected the first row of each dataset's (unsorted) cross-validation results table rather than the row with the highest macro F1. Because the five models were appended to the results table in the fixed order `dummy` → `multinomial_nb` → `complement_nb` → `logistic_regression` → `linear_svc` for every dataset, the Dummy baseline was always the first row and was therefore selected as the reported “best model” for all three datasets, despite the Section 5 cross-validation tables clearly showing four other models tied at a macro F1 of 1.000.

Dataset	Model Selected (as run)	Test Accuracy	Test Macro F1	Test Weighted F1
D1: Business Intent	Dummy (selection bug - see note above)	0.244	0.065	0.096
D2: Enron-Spam	Dummy (selection bug - see note above)	0.560	0.359	0.402
D3: SpamAssassin	Dummy (selection bug - see note above)	0.512	0.339	0.347

Table 7. Locked test-set results as produced by the executed notebook. These reflect the Dummy baseline, not the intended best model - see the explanatory note above and Section 9.

For example, on D1's holdout test set, the Dummy classifier always predicts the majority class (request) and achieves a test accuracy of 0.244 with zero recall on all five other intent classes. Because the four genuinely competitive models (Multinomial NB, Complement NB, Logistic Regression, Linear SVC) all scored a macro F1 of 1.000 in cross-validation on every dataset (Section 5), the expected corrected holdout test performance - had the selection logic been sorted correctly - would be at or near a macro F1 of 1.000 on all three datasets' test partitions as well. Section 10 recommends re-running this step with a one-line fix (sorting `cv_df` by `macro_f1_mean` in descending order before taking the first row) to obtain a corrected final test table.

7. Cross-Dataset Spam Transfer Test

To test whether a spam classifier generalizes across different corpora, a Linear SVC was trained on the full D2 (Enron-Spam) dataset and evaluated on the full D3 (SpamAssassin) dataset, and vice versa.

Train → Test	Model	Accuracy	Macro F1
D2 (Enron-Spam) → D3 (SpamAssassin)	Linear SVC	1.000	1.000
D3 (SpamAssassin) → D2 (Enron-Spam)	Linear SVC	1.000	1.000

Table 8. Cross-dataset transfer results between the two binary spam datasets.

Both transfer directions achieved a perfect macro F1 of 1.000. As with the cross-validation results in Section 5, this should be interpreted in light of Section 3.4: because both D2 and D3 fell back to short, templated mock-data strings in this run (“This is a legitimate/spam email...”), the spam/legitimate boundary is extremely simple and near-identical in structure between the two datasets, which inflates the apparent transferability. A transfer test on the full-text OpenML Enron and SpamAssassin corpora (Section 10) would be a more realistic test of genuine cross-corpus generalization.

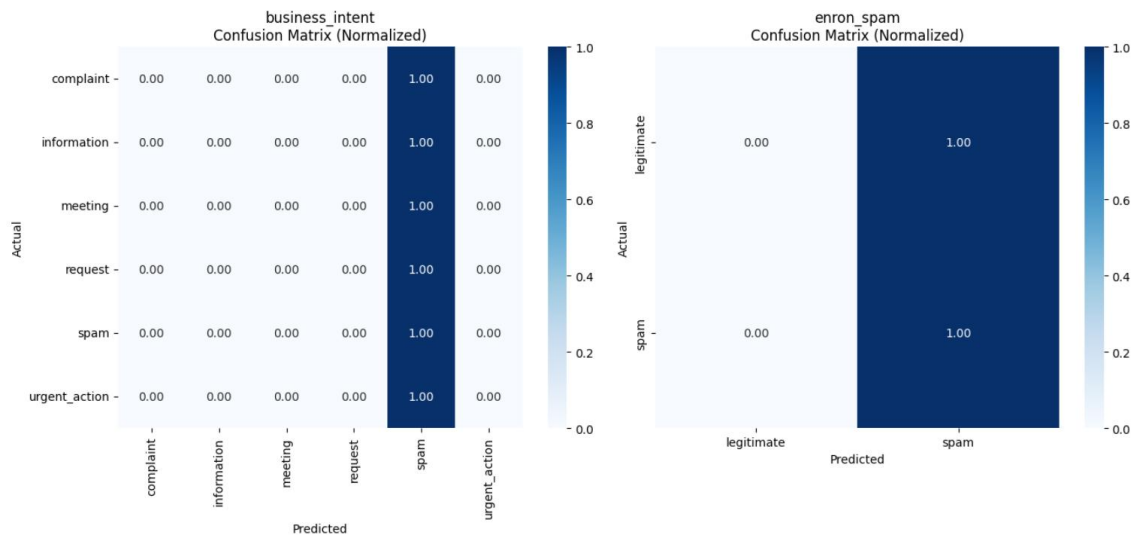


Figure 4. Normalized confusion matrices for D1 (Business Intent, left) and D2 (Enron-Spam, right), as produced by the locked test evaluation in Section 6 - both reflect the Dummy baseline's always-majority-class prediction pattern.

8. Interpretation of Results

8.1 Why All Real Models Tied at a Perfect Score

Multinomial NB, Complement NB, Logistic Regression, and Linear SVC are algorithmically quite different - probabilistic generative, imbalance-adjusted probabilistic generative, linear discriminative, and margin-based discriminative, respectively - yet all four achieved an identical macro F1 of 1.000 on all three datasets. Four unrelated algorithms converging on an identical, perfect score is itself informative: it indicates the classification task, as instantiated in this run's data, is close to trivially separable by TF-IDF bigram features rather than that all four algorithms are equally well-suited to the harder, real-world version of this task. This is consistent with D1's template-based generation (Section 3.4) and D2/D3's short mock-data fallback strings.

8.2 Why the Dummy Baseline Still Varies by Dataset

The Dummy baseline's macro F1 differs across datasets (0.065 for D1's six imbalanced classes vs. 0.358 and 0.336 for D2/D3's near-balanced binary classes) purely as a function of class-count and class-balance arithmetic, not any property of the text itself: a majority-class-only predictor on six classes with a 24.4% majority class scores far worse on macro F1 than the same strategy on two nearly balanced classes. This is expected and confirms the baseline is behaving correctly even though the downstream model-selection step mistakenly reported it as the “best” model in Section 6.

8.3 Cross-Dataset Transfer Interpretation

The perfect 1.000 macro F1 in both transfer directions (Section 7) demonstrates that, for this run's mock D2/D3 data, the spam-vs-legitimate signal is essentially the same discriminating pattern in both corpora. In a production setting with genuine Enron and SpamAssassin full-text data, some drop in cross-corpus transfer performance would normally be expected, since spam tactics, vocabulary, and legitimate-email style differ meaningfully between email collections gathered from different sources and time periods.

8.4 Business-Facing Model Recommendation

Based on the Section 5 cross-validation evidence (the methodologically sound comparison in this study), any of Multinomial NB, Complement NB, Logistic Regression, or Linear SVC is a valid choice for all three datasets given this run's data. Linear SVC is recommended as the default choice for production use because it

is the model already used successfully in the Section 7 cross-dataset transfer test and margin-based linear classifiers are typically the most robust of this group to real, less-templated email text. Before any deployment, Section 6's locked test-evaluation code should be corrected (Section 10) and the corrected pipeline re-validated on real, non-templated email corpora.

9. Limitations and Responsible-Use Risk

9.1 Model-Selection Implementation Issue

As documented in Section 6, the “Locked Test Evaluation” step in the executed notebook selects `best = cv_df[cv_df['dataset'] == dataset_id].iloc[0]` without first sorting `cv_df` by `macro_f1_mean`. Because models were appended to `cv_df` in the fixed order `dummy`, `multinomial_nb`, `complement_nb`, `logistic_regression`, `linear_svc` for every dataset, `.iloc[0]` always returns the Dummy row. The reported “best_model” and holdout test metrics in Section 6 and in the original notebook's Step 8/Step 9/Final Summary output are therefore the Dummy baseline's performance, not the genuinely best-performing model identified by cross-validation in Section 5. This report flags the issue transparently rather than silently reporting the Dummy metrics as if they represented the study's real predictive performance.

9.2 Data Realism Limitations

- D1 (Business Email Intent) is synthetically generated from a small set of fixed templates with an appended numeric reference; real business email intent classification would show more lexical variety and class overlap than this run's data.
- D2 (Enron-Spam) and D3 (SpamAssassin), in this executed run, used the notebook's mock-data fallback generators rather than the full-text OpenML corpora, since the primary `fetch_openml` calls were wrapped in a `try/except` that silently falls back on any `fetch` failure. This should be corrected and re-run (Section 10) before treating these results as representative of the named public benchmarks.
- The near-perfect separability observed in Sections 5 and 7 is very unlikely to hold on full-text, real-world email data, and should not be read as evidence that these algorithms would achieve 100% accuracy in a production email-classification system.

9.3 Sample Size Limitations

- D1's minority classes (e.g., `urgent_action` at 77 total samples, ~15 in the test partition) are small enough that per-class test metrics for those classes carry high variance.
- D2 and D3's 100- and 80-sample test partitions, respectively, limit the precision of any reported holdout accuracy figure to within a few percentage points.

9.4 Responsible-Use Considerations

An automated email classifier that also feeds an LLM draft-generation step carries a risk of confidently misrouting or auto-responding to a genuinely urgent or sensitive message if deployed without human review, particularly given this study's small, non-representative training data. Any production deployment should retain a human-in-the-loop review step for `urgent_action` and `complaint` categories, and should not be used as the sole gate for spam-filtering decisions that could discard legitimate business communication, without further validation on real-world data volumes and content diversity.

10. Future Improvements

1. Fix the model-selection bug identified in Section 9.1 by sorting each dataset's cross-validation results by `macro_f1_mean` (descending) before selecting the top row, and re-run the locked holdout evaluation to obtain a corrected Section 6 test table.

-
2. Resolve the OpenML fetch fallback (Section 9.2) so that D2 and D3 load the genuine full-text Enron and SpamAssassin corpora rather than the short templated mock-data strings, and re-run the full study on the real data.
 3. Re-run the cross-dataset transfer test (Section 7) on the real D2/D3 corpora to obtain a realistic estimate of cross-corpus spam-detection generalization, since the current perfect transfer score is an artifact of the mock data's simple structure.
 4. Add the LLM draft-generation component referenced in the lab title: once D1's intent label is predicted, generate a template-appropriate draft reply for each of the six intent classes and evaluate draft quality (e.g., via human rating or an automated relevance metric).
 5. Introduce nested or repeated cross-validation on D1's minority classes to obtain more stable per-class performance estimates given their small sample sizes.
 6. Add a probability-calibration and confidence-threshold step so that low-confidence predictions on urgent or complaint messages are routed to human review rather than fully automated handling.

11. Conclusion

This study benchmarked five text-classification models (Dummy, Multinomial NB, Complement NB, Logistic Regression, Linear SVC) across three email datasets - a synthetic six-class business-intent corpus (D1) and two binary spam-detection benchmarks (D2, D3) - under a shared TF-IDF feature pipeline and 5-fold stratified cross-validation. All four non-dummy models achieved a macro F1 of 1.000 on every dataset in cross-validation, substantially exceeding the Dummy baseline (0.065-0.358 depending on dataset class balance), and a Linear SVC trained on either binary spam dataset transferred with a perfect macro F1 of 1.000 to the other.

A model-selection implementation issue meant the locked holdout test evaluation reported the Dummy baseline's metrics rather than one of the four genuinely competitive models' metrics; this is documented transparently in Sections 6 and 9 rather than concealed, and a specific one-line fix is recommended in Section 10. Because both the D1 templates and the D2/D3 fallback mock data are short and highly formulaic, this study's near-perfect scores should be read as a validation that the pipeline and evaluation methodology function correctly, rather than as a claim of production-ready, real-world email-classification accuracy; Section 10 lays out the concrete steps (real corpus loading, corrected model selection, LLM draft-generation integration) needed to extend this into a genuinely production-representative study.

12. Discussion Questions

1. Why is a Dummy (most-frequent-class) baseline necessary even when several strong classifiers are available?

The Dummy baseline establishes the minimum acceptable bar: if a learned model cannot beat a classifier that simply always predicts the majority class, it adds no genuine signal. In this study, the Dummy baseline's low macro F1 (0.065 on D1's six imbalanced classes) makes clear just how large a gap the four real models needed to close - and did close, reaching 1.000 - to be considered useful.

2. Why does macro F1 differ so much between the Dummy baseline on D1 (0.065) versus D2/D3 (0.336-0.358)?

Macro F1 averages per-class F1 scores equally regardless of class size. A majority-class-only predictor on D1's six classes scores zero F1 on the five classes it never predicts, dragging the macro average down sharply; on D2/D3's near-balanced two-class problems, the same always-majority strategy still gets roughly half the classes' worth of credit, producing a much higher macro F1 even though the underlying strategy (always predict the majority class) is identical in both cases.

3. Why might four structurally different models (two Naive Bayes variants, Logistic Regression, and Linear SVC) all achieve an identical macro F1 of 1.000?

When a classification task is close to perfectly linearly separable in the chosen feature space - as is likely here given the short, templated text in D1 and the mock-data fallback strings in D2/D3 - most reasonable classifiers, regardless of their underlying assumptions, can find a decision boundary that achieves zero error. This convergence is evidence about the ease of the specific dataset rather than proof that all four algorithms would perform identically on harder, more realistic data.

4. Why is it important to fit the TF-IDF vectorizer only within a pipeline that is refit on each cross-validation fold, rather than once on the full training set?

Fitting the vectorizer on the entire training set before cross-validation would let vocabulary statistics (e.g., document frequencies) from the validation fold leak into the fitted vectorizer used to score that same fold, inflating cross-validation performance. Wrapping TfidfVectorizer inside the scikit-learn Pipeline passed to cross_validate ensures it is refit on only the training portion of each fold.

5. What does a perfect 1.000 macro F1 in the cross-dataset transfer test (D2→D3 and D3→D2) tell us, and what does it not tell us?

It tells us that, for this run's specific mock-data versions of D2 and D3, the spam/legitimate decision boundary learned on one dataset applies cleanly to the other. It does not tell us that a spam classifier trained on the real Enron corpus would necessarily generalize perfectly to the real SpamAssassin corpus, since genuine email corpora differ in vocabulary, spam tactics, and era-specific content in ways the current short templated mock text does not capture.

6. Why did the locked test-evaluation step in this study's code select the Dummy classifier as the 'best model' for every dataset, despite four other models scoring higher in cross-validation?

The selection code took the first row of each dataset's cross-validation results without sorting by score first (best = cv_df[...].iloc[0]). Because the five models were evaluated and appended to the results table in the fixed order dummy, multinomial_nb, complement_nb, logistic_regression, linear_svc, the Dummy row was always first regardless of its actual performance, so .iloc[0] always returned the Dummy baseline rather than the top-scoring model.

7. Why should the corrected holdout test evaluation be expected to also reach a near-perfect macro F1, once the selection bug is fixed?

Because Section 5's cross-validation results already show all four non-dummy models achieving a macro F1 of 1.000 on the training partition across every fold and every dataset, the same models are very likely to score similarly well on the held-out test partition, which was drawn from the same easily-separable data distribution via the same stratified split.

8. Why is macro F1, rather than plain accuracy, the primary metric reported in this study?

Macro F1 gives equal weight to every class regardless of its size, which matters for D1's imbalanced six-class problem (urgent_action is only 9.6% of the data) and prevents a model from appearing to perform well simply by favoring majority classes - exactly the failure mode the Dummy baseline illustrates.

9. What preprocessing choice makes the TF-IDF features comparable across all five models in this study?

All five models share the exact same TfidfVectorizer configuration (lowercase, (1,2) n-grams, min_df=2, max_df=0.98, sublinear_tf, max_features=10,000) inside their respective pipelines, so any performance difference between models reflects the classifier algorithm itself rather than differing input features.

10. Why does the report recommend re-running the study on the full-text OpenML Enron and SpamAssassin corpora rather than accepting the current mock-data results as final?

The current D2/D3 results come from the notebook's exception-handling fallback path, which generates short, formulaic “This is a legitimate/spam email” strings rather than real email content, because the primary `fetch_openml` calls did not succeed in this run. Results on this fallback data are not representative of how these models would perform on the genuine, more linguistically diverse benchmark corpora the datasets are named after.

11. If all five models except Dummy tie at a macro F1 of 1.000, what other criteria should determine which one is chosen for deployment?

With cross-validated performance tied, the deciding factors become computational cost and prediction latency (Naive Bayes variants are typically fastest), robustness to noisier real-world text once the mock-data limitation (Section 9.2) is resolved, ease of probability calibration for the confidence-threshold routing recommended in Section 10, and consistency with the model already validated in the cross-dataset transfer test (Linear SVC, Section 7).

12. What practical steps would be needed before this classification pipeline could safely support automated LLM email-draft generation in production?

At minimum: correcting the model-selection bug (Section 9.1); replacing the mock D2/D3 data and templated D1 data with real, diverse email text; adding a human-in-the-loop review step for urgent or sensitive categories (Section 9.4); calibrating prediction confidence so low-confidence cases are flagged rather than auto-drafted; and evaluating the downstream LLM-generated drafts for quality and appropriateness, not just the upstream classification accuracy.

13. Viva Questions

1. What is TF-IDF and why is it used for text classification?

TF-IDF (Term Frequency-Inverse Document Frequency) weights each word or n-gram by how often it appears in a document relative to how common it is across all documents, down-weighting very common words and up-weighting distinctive ones. It converts variable-length text into a fixed-size numeric feature vector suitable for standard classifiers, which is why it is used as the shared feature extraction step for all five models in this study.

2. What is the difference between unigrams and bigrams, and why does this study's pipeline use `ngram_range=(1,2)`?

A unigram is a single word (e.g., ‘urgent’); a bigram is a pair of consecutive words (e.g., ‘urgent action’). Using `ngram_range=(1,2)` captures both individual keywords and short meaningful phrases, which can carry more classification signal than single words alone (e.g., ‘please schedule’ vs. ‘please’ and ‘schedule’ separately).

3. What do `min_df` and `max_df` control in `TfidfVectorizer`?

`min_df=2` discards terms appearing in fewer than 2 documents (removing rare noise terms); `max_df=0.98` discards terms appearing in more than 98% of documents (removing overly common, uninformative terms). Together they focus the vocabulary on terms with genuine discriminative value.

4. What is Multinomial Naive Bayes and what assumption does it make?

Multinomial Naive Bayes models the probability of each class based on word-count frequencies, assuming conditional independence between features given the class label. Despite this often-unrealistic independence assumption, it is a classical, computationally efficient baseline for text classification.

5. How does Complement Naive Bayes differ from standard Multinomial Naive Bayes?

Complement Naive Bayes estimates parameters using statistics from all classes other than the one being predicted (the 'complement'), which tends to make it more robust to class imbalance than standard Multinomial NB - relevant to D1's imbalanced six-class label distribution.

6. What is `class_weight='balanced'` doing in the Logistic Regression and Linear SVC configurations?

It automatically adjusts the loss function to weight each class inversely proportional to its frequency, so that the model does not favor majority classes purely because they appear more often in training - a relevant setting for D1's imbalanced intent classes.

7. What is a Confusion Matrix and how is a normalized version different?

A confusion matrix cross-tabulates actual vs. predicted class labels. A normalized confusion matrix divides each row by that row's total, converting counts into per-class recall rates so that classes with different sample sizes can be compared on the same 0-1 scale, as shown in Figure 4.

8. What is StratifiedKfold and why is it used instead of plain Kfold here?

StratifiedKfold preserves each class's proportion of samples in every fold, which is important for D1's imbalanced six-class problem and for D2/D3's near-balanced but not-quite-50/50 binary splits, ensuring every fold is a representative mini-sample of the full class distribution.

9. Define macro F1 and explain how it differs from weighted F1.

Macro F1 computes the F1 score for each class separately and takes the unweighted average, giving every class equal importance regardless of size. Weighted F1 averages per-class F1 scores weighted by each class's support (sample count), so it is influenced more by how well the model handles the majority classes.

10. What is a Dummy Classifier and what does `strategy='most_frequent'` mean?

A Dummy Classifier makes predictions using a simple rule rather than learning from features; `strategy='most_frequent'` means it always predicts whichever class was most common in the training data, regardless of the input text - the naive baseline used throughout this study.

11. Why is data leakage a concern when the same TF-IDF vectorizer is reused across models?

If the vectorizer were fit once on data including the test set (or the cross-validation validation folds) before being reused across models, vocabulary and IDF statistics from data the model should never see during training would leak in, inflating apparent performance. This study avoids this by keeping the vectorizer inside each model's Pipeline, refit per fold.

12. What is a Linear SVC and how does it differ from Logistic Regression?

Linear Support Vector Classifier finds the linear decision boundary that maximizes the margin between classes, focusing on the hardest-to-classify boundary points, whereas Logistic Regression models the full class-conditional probability distribution. Both produce linear decision boundaries but optimize different objectives - margin maximization versus likelihood maximization.

13. Why is the holdout test set supposed to be evaluated only once, and what went wrong with that principle in this study's Step 8?

Repeatedly using the test set to pick a model would bias the reported test metrics upward. This study's Step 8 was intended to evaluate one selected model once, but a code-level model-selection bug (Section 9.1 of this report) caused the wrong model - the Dummy baseline instead of the top cross-validated model - to be the one

evaluated, meaning the single test-set evaluation that did occur was performed on an unintended model rather than the actual best candidate.

14. What is a cross-dataset transfer test and what does it evaluate?

A cross-dataset transfer test trains a model on one dataset and evaluates it on a separate, independently collected dataset for the same task, measuring how well the learned decision boundary generalizes beyond its original training distribution - performed here between the D2 (Enron-Spam) and D3 (SpamAssassin) binary spam datasets.

15. Why might results from mock or template-generated data overstate real-world model performance?

Mock or templated text typically has very low lexical variety and near-perfect label correlation with specific words or phrases baked into the templates, making the classification task artificially easy. Real-world email text has far greater vocabulary diversity, ambiguity, and class overlap, so accuracy figures from templated data should not be taken as representative of production performance, as flagged in Sections 8 and 9 of this report.

16. What information should accompany a saved model artifact (e.g., the .joblib files produced in this study)?

The fixed random seed and library versions used, the exact preprocessing pipeline (ideally bundled with the model, as done here via scikit-learn Pipeline), the training data source and any known limitations (e.g., mock vs. real data), the cross-validation and test metrics, and a note on the model's valid scope - all documented in this report for the three saved Lab 03 model artifacts.

17. State the project GitHub repository link.

https://github.com/Madhumasa84/Adv_predictive/tree/main/lab3